



Tutorial 02: Implementasi Model-View-Template (MVT) pada Django

Pemrograman Berbasis Platform (CSGE602022) - diselenggarakan oleh Fakultas Ilmu Komputer Universitas Indonesia, Semester Gasal 2026/2027

Kontributor: FEIN - Alvin Christian Halim, REM - Malik Alifan Kareem, TIM - Justin Timothy W

Turunkan Versi Django Kalau Deployment Gagal

Kalau *deployment* ke PWS gagal karena masalah versi Django, turunkan versinya dengan mengubah baris `django` di `requirements.txt` menjadi:

```
django~=5.0
```

Lalu *push* ulang ke PWS.

Tujuan Pembelajaran

Setelah menyelesaikan tutorial ini, mahasiswa diharapkan untuk dapat:

- Mengerti konsep **MVT** pada aplikasi Django
- Mengerti bagaimana **alur Django menampilkan sebuah halaman HTML**
- Mengerti konfigurasi *routing* yang ada pada `urls.py`
- Memahami kaitan *models*, *views* dan *template* pada Django
- Memahami pembuatan *unit test* pada *framework* Django

Peringatan

Tutorial ini adalah prasyarat wajib untuk Individual Assignment minggu ini - jika Tutorial ini belum diselesaikan, Individual Assignment tersebut tidak akan dinilai. Deadline Tutorial 02 adalah **Rabu, 9 September 2026**; deadline Individual Assignment 2 adalah **Senin, 14 September 2026**. Lihat [halaman Jadwal](#) untuk tanggal lengkap.

Tutorial ini adalah **Tutorial 02**, lanjutan langsung dari [Tutorial 01](#) - tutorial ketiga dari proyek yang akan kamu bangun berkelanjutan sepanjang semester: sebuah **website portofolio pribadi**. Semua tutorial berikutnya akan melanjutkan langsung dari hasil tutorial ini, jadi pastikan proyekmu berjalan dengan baik sebelum lanjut ke tutorial berikutnya.

Sepanjang tutorial ini, contoh yang dipakai adalah portofolio milik **Burhan**, maskot mata kuliah PBP. Setiap kali ada bagian yang berisi data pribadi Burhan (nama, NPM, bio, foto), akan ada catatan eksplisit yang bilang "ganti ini dengan data kamu sendiri" - jangan sampai kelewat.

Tip

Yang Sudah Kita Bangun Sejauh Ini - Di Tutorial 1, kamu mengganti halaman bawaan Django dengan halaman portofolio "About Me", menghubungkan *view* ke *template* HTML, menambahkan CSS, dan men-*deploy* hasilnya ke PWS.

Pada tutorial ini, kita akan memindahkan data profil yang masih ditulis langsung di HTML ke dalam *context* dan membuat halaman **Experience** yang mengambil data dari model. Halaman profil tetap di `/`, sedangkan daftar pengalaman akan tampil di `/experience/`.

Struktur folder proyekmu saat ini (hasil akhir Tutorial 1):

```
myportofolio
├── env/
├── .git/
├── .gitignore
├── requirements.txt
├── db.sqlite3
├── manage.py
├── portofolio                # folder konfigurasi utama proyek
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py          # semua konfigurasi proyek ada di sini
│   ├── urls.py              # daftar routing URL
│   ├── views.py
│   └── wsgi.py
├── static
│   ├── css
│   │   └── style.css
│   └── img
│       └── burhan.png
└── templates
    └── index.html
```

Pengenalan Konsep MVT (Model-View-Template)

Dalam dunia *web development*, terdapat berbagai pola arsitektur yang dapat membantu programmer dalam merancang dan mengembangkan sebuah aplikasi. Contohnya ada MVC (Model-View-Controller), MVT (Model-View-Template), dan masih banyak lagi. Pada kuliah PBP, kita akan lebih terfokus dalam MVT yang digunakan oleh Django.

Apa Itu Konsep MVT?

Seperti yang sudah kalian lihat, MVT terdiri atas Model, View, dan Template. Secara sederhana:

- Model merupakan data & logika bisnis
- Template merupakan tampilan akhir yang dilihat oleh pengguna

- View merupakan jembatan yang menghubungkan antara data(model) dan tampilan(template)

Apa Itu Model?

Model bertugas untuk **mengatur** dan **mengelola** data pada sebuah aplikasi. Pada Django sendiri, sudah disediakan *Object Relational Mapping(ORM)* sehingga kamu bisa bekerja dengan command python, tanpa SQL langsung seperti pada database umumnya.

Contoh model:

```
class Experience(models.Model):
    title = models.CharField(max_length=255)
    description = models.TextField()
    started_at = models.DateField(auto_now_add=True)
    ended_at = models.DateField(null=True, blank=True)
```

Apa Itu View?

View menangani logika yang akan ditampilkan kepada pengguna. Alur view meliputi penerimaan request dari pengguna, mengambil data dari model, lalu mengirimkannya ke template untuk ditampilkan ke pengguna.

Contoh view:

```
def show_experience(request):
    experience_list = Experience.objects.all()
    return render(request, "experience.html", {"experience_list": experience_list})
```

Apa Itu Template?

Template merupakan berkas HTML yang menentukan bagaimana aplikasi ditampilkan.

Contoh template:

```
<h1>Experiences</h1>
<ul>
    {% for experience in experience_list %}
        <li>
            <strong>{{ experience.title }}</strong>
            <br>
            <small>
                {{ experience.started_at }} – {{ experience.ended_at }}
            </small>
        </li>
    {% endfor %}
</ul>
```

Alur MVT

1. Pengguna mengakses URL tertentu melalui browser.
2. Django akan memetakan URL tersebut ke View melalui `urls.py`.
3. View mengambil dan memproses data dari Model apabila dibutuhkan.

4. View mengirim *context*-nya ke Template.
5. Django mengembalikan HTML yang telah dirender sebagai *response*.

Catatan

Alurnya secara simple: Request → urls → View → Model → View → Template → Response

Manfaat MVT

Manfaat MVT ada banyak, di antaranya:

1. **Pemisahan Tugas** - Logika aplikasi, tampilan, dan data dipisahkan secara jelas dan rapi sehingga lebih mudah dikelola untuk developer.
2. **Kode yang Lebih Terstruktur** - Aplikasi menjadi lebih modular, mudah diuji dengan test, dan juga scalable.
3. **Reusability** - Template & View dapat digunakan kembali di berbagai bagian aplikasi secara berulang.

Pre-Tutorial Notes

Peringatan

Sebelum memulai tutorial ini, mohon periksa kembali berkas `requirements.txt` di proyek portofolimu. Baris *dependency* Django kemungkinan masih tertulis:

```
django~=6.0
```

(atau `django` tanpa batasan versi).

Harap ubah baris tersebut menjadi **Django versi 5.2**:

```
django~=5.2
```

Lalu, pastikan *virtual environment* sudah aktif dan jalankan instalasi ulang:

```
pip install -r requirements.txt
```

Hal ini perlu dilakukan karena database PostgreSQL yang digunakan PWS belum mendukung Django versi terbaru (versi 6.x). Oleh karena itu, versi Django yang digunakan perlu diturunkan ke versi 5.2 yang merupakan versi LTS terbaru saat ini.

Jika tidak sengaja push file sensitif seperti `.env`, `.env.prod`, `db.sqlite3`, atau folder `env/`, hapus dari Git menggunakan:

```
git rm --cached .env
git rm --cached .env.prod
git rm --cached db.sqlite3
git rm -r --cached env/
```

Catatan

Jika muncul pesan `fatal: pathspec did not match any files`, kamu bisa mengabaikannya. Pesan tersebut menandakan berkas atau folder yang ingin kamu hapus memang belum pernah dicatat oleh Git di repositori lokal milikmu.

Catatan: Perintah di atas menghapus file sensitif dari tracking Git ke depannya.

Cek apakah `.gitignore` sudah berisi file sensitif tersebut:

```
cat .gitignore
```

Jika belum ada, tambahkan ke `.gitignore`:

```
.env*  
db.sqlite3  
env/
```

Kemudian buat commit clean up:

```
git add .  
git commit -m "cleanup"
```

Dengan demikian, kita dapat meminimalisasi risiko keamanan dari credential yang ter-expose di repository publik.

Tutorial: Membuat Aplikasi Django beserta Konfigurasi Model

Dalam tutorial ini, akan dijelaskan mengenai konsep aplikasi dan proyek dalam Django.

Apa Itu Proyek dan Aplikasi dalam Django?

- **Proyek (Project)** adalah keseluruhan proyek web yang kamu bangun dengan menggunakan Django. **Proyek berisi berbagai aplikasi** yang berfungsi secara bersama untuk menciptakan situs web atau aplikasi web yang lengkap.
- **Aplikasi (Apps)** adalah unit modular yang melakukan tugas-tugas spesifik dalam suatu proyek Django. Setiap aplikasi dapat memiliki model, tampilan, *template*, dan URL yang terkait dengannya. Aplikasi memungkinkanmu untuk membagi fungsionalitas proyek menjadi bagian-bagian terpisah yang dapat dikelola secara independen.

Catatan

Sebelum dimulai, kamu perlu mengingat kembali bahwa direktori utama adalah direktori **terluar** (`myportfolio`), sedangkan direktori proyek adalah direktori **di dalam** direktori utama (`portfolio`).

Langkah 1: Persiapan Awal

1. Buka Direktori Utama `myportfolio`.

- Sebelum memulai, pastikan kamu berada di direktori **utama** `myportofolio` yang telah dibuat pada Tutorial 0 dan dilanjutkan pada Tutorial 1.
- Pengembangan proyek Django kamu pada tutorial sebelumnya akan dilanjutkan pada direktori ini 😊.

2. Buka terminal atau *command prompt* dan pastikan kamu sudah berada pada direktori utama, `myportofolio`.

Catatan

Gunakan perintah `cd [direktori]` untuk berpindah direktori ke direktori lain yang ingin dituju. Perintah ini sangat penting untuk diingat, karena keahlian menggunakan terminal akan bermanfaat tidak hanya untuk mata kuliah PBP, tetapi juga mata kuliah lain nantinya.

1. Aktifkan *virtual environment* yang telah dibuat sebelumnya dengan menjalankan perintah berikut. **(Mohon perhatikan sistem operasi yang kamu gunakan).**

- **Windows:**

```
env\Scripts\activate
```

- **Unix (Linux & Mac OS):**

```
source env/bin/activate
```

Catatan

- Untuk pengguna Windows, apabila kamu mendapatkan error berbunyi "*the execution of scripts is disabled on this system...*", berikut adalah solusi yang dapat kamu coba:
 - Buka *PowerShell* sebagai *administrator* dan jalankan perintah berikut:
- Untuk pengguna sistem operasi berbasis Unix (Linux & macOS), jika kamu mendapatkan error berbunyi "... *Permission Denied*", berikut adalah solusi yang dapat kamu coba:
 - Jalankan perintah berikut:

```
Set-ExecutionPolicy Unrestricted -Force
```

```
chmod +x env/bin/activate
```

Langkah 2: Membuat Aplikasi `main` dalam Proyek `myportofolio`

Kamu akan membuat aplikasi baru bernama `main` dalam proyek `myportofolio`.

1. Jalankan perintah berikut untuk membuat aplikasi baru dengan nama **main**.

```
python manage.py startapp main
```

Setelah perintah di atas dijalankan, direktori baru dengan nama `main` akan terbentuk. Direktori `main` akan berisi struktur awal untuk aplikasi Django kamu.

Catatan

Jika kamu masih bingung mengenai istilah-istilah baru seperti **direktori utama**, **direktori proyek**, **direktori aplikasi**, *it's okay!* Kamu akan terbiasa seiring berjalannya waktu. Semangat!

2. Mendaftarkan aplikasi `main` ke dalam proyek.

- Buka berkas `settings.py` di dalam direktori proyek `portofolio`.
- Tambahkan `'main'` ke dalam daftar aplikasi yang ada sebagai elemen paling terakhir. Daftar aplikasi dapat kamu akses pada variabel `INSTALLED_APPS`.

```
INSTALLED_APPS = [  
    ...,  
    'main'  
]
```

Dengan melakukan langkah-langkah tersebut, kamu telah mendaftar aplikasi `main` ke dalam proyek `myportofolio` kamu.

Tutorial: Implementasi Model Dasar

Langkah 1: Mengubah Berkas `models.py` dalam Aplikasi `main`

Pada langkah ini, kamu akan mengubah berkas `models.py` yang terdapat di dalam direktori aplikasi `main` untuk mendefinisikan model baru.

1. Buka berkas `models.py` pada direktori aplikasi `main`.
2. Isi berkas `models.py` dengan kode berikut.

```
import uuid  
from django.db import models  
  
class Experience(models.Model):  
    EXPERIENCE_CHOICES = [  
        ('internship', 'Internship'),  
        ('research', 'Research'),  
        ('volunteer', 'Volunteer'),  
        ('part-time', 'Part-Time'),  
        ('full-time', 'Full-Time'),  
        ('freelance', 'Freelance'),  
    ]  
  
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)  
    title = models.CharField(max_length=255)  
    description = models.TextField()  
    category = models.CharField(max_length=20, choices=EXPERIENCE_CHOICES, default='full-time')
```

```

thumbnail = models.URLField(blank=True, null=True)
started_at = models.DateTimeField(auto_now_add=True)
ended_at = models.DateTimeField(blank=True, null=True)
def __str__(self):
    return self.title

@property
def is_ongoing(self):
    return self.ended_at is None

```

Penjelasan Kode:

- `models.Model` adalah kelas dasar yang digunakan untuk mendefinisikan model dalam Django.
- `Experience` adalah nama model yang kamu definisikan.
- `EXPERIENCE_CHOICES` adalah tuple yang mendefinisikan pilihan kategori pengalaman yang tersedia.
- `id` adalah field bertipe `UUIDField` yang digunakan sebagai primary key dan nilainya di-generate otomatis menggunakan `uuid.uuid4`.
- `title` adalah field bertipe `CharField` untuk judul pengalaman, dengan panjang maksimal 255 karakter.
- `description` adalah field bertipe `TextField` untuk deskripsi pengalaman yang dapat menampung teks panjang.
- `category` adalah field bertipe `CharField` dengan pilihan terbatas sesuai `EXPERIENCE_CHOICES`, dengan nilai default `'full-time'`.
- `thumbnail` adalah field bertipe `URLField` untuk menyimpan URL gambar thumbnail pengalaman (opsional).
- `started_at` adalah field bertipe `DateTimeField` yang otomatis berisi tanggal dan waktu saat data dibuat.
- `ended_at` adalah field bertipe `DateTimeField` yang dapat dibiarkan kosong dan nilainya dapat diatur ke `None`.
- Method `__str__` digunakan untuk mengembalikan representasi string dari objek (dalam hal ini judul pengalaman).
- Decorator `@property` digunakan untuk membuat atribut read-only yang nilainya merupakan hasil perhitungan dari atribut lain. Dalam kasus ini, `is_ongoing` akan bernilai `True` jika `ended_at` adalah `None`.

Catatan

Kamu akan mempelajari lebih banyak tentang “atribut hasil derivasi” pada mata kuliah Basis Data nanti. Untuk sementara, apabila kamu ingin mengetahui lebih banyak tentang kegunaan decorator `@property`, kamu dapat membaca [dokumentasi Python mengenai class property](#).

Catatan

Menurut [dokumentasi resmi Django](#), penggunaan `null=True` sebaiknya dihindari pada *field* berbasis string seperti `CharField`, `TextField`, dan `URLField`. Hal ini karena nilai kosong pada *field* string direpresentasikan sebagai string kosong `""` menurut konvensi *framework* Django. Penggunaan `null=True` akan menyebabkan ada dua kemungkinan status data kosong di basis data (`NULL` dan `""`).

Pada tutorial ini, `thumbnail` menggunakan `null=True` sebagai contoh. Namun untuk tugas dan proyek mandiri selanjutnya, kamu sebaiknya menggunakan `blank=True` (atau `blank=True, default=""`). Jika kamu sudah

terlanjur menjalankan migrasi sesuai kode di atas, kamu tidak perlu mengubahnya atau melakukan migrasi ulang karena keduanya tetap berfungsi dengan baik pada tutorial ini.

Langkah 2: Membuat dan Mengaplikasikan Migrasi Model

Apa itu migrasi model?

- Migrasi model adalah cara Django melacak perubahan pada model basis data kamu.
- Migrasi ini adalah instruksi untuk mengubah struktur tabel basis data sesuai dengan perubahan model yang didefinisikan dalam kode terbaru kamu.

Bagaimana cara melakukan migrasi model?

1. Jalankan perintah berikut untuk membuat migrasi model.

```
python manage.py makemigrations
```

Catatan

`makemigrations` menciptakan berkas migrasi yang berisi perubahan model yang **belum** diaplikasikan ke dalam basis data.

2. Jalankan perintah berikut untuk menerapkan migrasi ke dalam basis data lokal.

```
python manage.py migrate
```

Catatan

`migrate` mengaplikasikan perubahan model yang tercantum dalam berkas migrasi ke basis data dengan menjalankan perintah sebelumnya.

Peringatan

Setiap kali kamu melakukan perubahan pada *model*, seperti menambahkan atau mengubah atribut, **kamu WAJIB melakukan migrasi** untuk merefleksikan perubahan tersebut.

Tutorial: Menghubungkan *View* dengan *Template*

Pada tahap ini, kamu akan membuat dua *view* pada aplikasi `main`: `show_main` untuk halaman profil dan `show_experience` untuk halaman pengalaman. Masing-masing *view* menyiapkan *context* dan meneruskannya ke *template* yang sesuai.

Langkah 1: Membuat Fungsi `show_main` dan `show_experience`

1. Buka berkas `main/views.py`.
2. Ganti isi berkas tersebut dengan kode berikut. Ubah nilai data profil Burhan menjadi data dirimu sendiri, termasuk `name` pada kedua *view*.

```
from django.shortcuts import render
```

```

from main.models import Experience

def show_main(request):
    context = {
        "name": "Burhan",
        "npm": "2206000000",
        "study_program": "S1 Ilmu Komputer",
        "bio": (
            "Mahasiswa Ilmu Komputer Universitas Indonesia yang tertarik "
            "pada pengembangan perangkat lunak dan pendidikan."
        ),
    }
    return render(request, "index.html", context)

def show_experience(request):
    context = {
        "name": "Burhan",
        "experience_list": Experience.objects.all(),
    }
    return render(request, "experience.html", context)

```

Penjelasan Kode:

- `Experience.objects.all()` mengambil seluruh objek `Experience` dari basis data dalam bentuk *QuerySet*.
- `context` adalah *dictionary* yang memetakan nama variabel dengan data yang akan tersedia pada *template*.
- `render(request, "index.html", context)` memproses `templates/index.html` menggunakan data dalam `context`, lalu mengembalikan respons HTML.
- `show_main` mengirim data profil ke `index.html`, sedangkan `show_experience` mengirim nama dan `experience_list` ke `experience.html`.
- Setiap view memiliki *context* sendiri. Nilai `name` pada `show_experience` dipakai oleh judul, *header*, dan *footer* halaman pengalaman.

Langkah 2: Menampilkan Data *Context* pada *Template*

Buka `templates/index.html`, lalu ganti data profil yang masih ditulis langsung dengan *template variable* berikut (baris ... menandakan ada konten lain di antaranya yang tidak perlu diubah):

```

<title>{{ name }} - Portofolio</title>
...
<a href="#profile" class="brand">{{ name }}</a>
...
<h1>{{ name }}</h1>
...

...
<p class="bio">{{ bio }}</p>
...
<dd>{{ npm }}</dd>

```

```
...
<dd>{{ study_program }}</dd>
```

Terakhir, ubah nama pada bagian *footer*:

```
<p>&copy; 2026 {{ name }}. Fakultas Ilmu Komputer, Universitas Indonesia.</p>
```

Sintaks `{{ name }}`, `{{ npm }}`, `{{ study_program }}`, dan `{{ bio }}` adalah *template variable* Django. Ketika halaman diproses, Django menggantinya dengan nilai dari `context` yang memiliki nama kunci yang sama.

Catatan

`index.html` tetap menjadi halaman profil. Pada langkah berikutnya, kita akan membuat `experience.html` untuk menampilkan daftar pengalaman. Selesaikan bagian *routing* sebelum mencoba kedua halaman melalui browser, karena *view* baru belum dihubungkan ke URL.

Langkah 3: Membuat Halaman Experience secara Dinamis

Sekarang data profil sudah berasal dari `context`. Kita akan membuat halaman terpisah yang memakai Django Template Language untuk mengubah setiap objek `Experience` menjadi satu kartu HTML.

1. Salin `templates/index.html` menjadi `templates/experience.html` di folder `templates/` yang sama, sejajar dengan `manage.py`. Folder ini sudah didaftarkan melalui `TEMPLATES.DIRS` pada Tutorial 1.
2. Di `experience.html`, ubah judul menjadi `<title>Experience - {{ name }}</title>`. Pertahankan `<head>`, tautan CSS, *header*, dan *footer* dari halaman profil agar tampilannya konsisten.
3. Ganti **seluruh elemen** `<main>...</main>` pada `experience.html` dengan kode berikut. Dengan demikian, bagian profil hanya ada di `index.html`.

```
<main>
  <section class="experience-section" id="experience">
    <div class="container">
      <p class="section-kicker">Perjalanan saya sejauh ini</p>
      <h1>Experience</h1>

      <div class="experience-grid">
        {% for experience in experience_list %}
          <article class="experience-card">
            <span class="experience-category">
              {{ experience.get_category_display }}
            </span>
            <h2>{{ experience.title }}</h2>
            <p class="experience-description">
              {{ experience.description }}
            </p>

            {% if experience.is_ongoing %}
              <p class="experience-status">Sedang berlangsung</p>
            {% else %}
              <p class="experience-status">Selesai</p>
            {% endif %}
          </article>
```

```

        {% empty %}
            <p class="empty-state">
                Belum ada pengalaman yang ditambahkan.
            </p>
        {% endfor %}
    </div>
</div>
</section>
</main>

```

Perhatikan tiga bentuk sintaks baru berikut:

- `{{ ... }}` menampilkan sebuah nilai, misalnya judul pengalaman.
- `{% ... %}` menjalankan logika *template*, misalnya perulangan atau percabangan.
- `{% empty %}` dijalankan ketika `experience_list` tidak memiliki data.

`get_category_display` mengubah nilai seperti `part-time` menjadi label yang lebih ramah dibaca, yaitu `Part-Time`. Method ini disediakan otomatis oleh Django untuk *field* yang menggunakan `choices`.

Catatan

Django *template* hanya mengatur tampilan. Pengambilan data tetap dilakukan oleh *view*. Pemisahan ini membuat HTML tidak perlu mengetahui cara basis data bekerja.

Kita akan menghubungkan navigasi kedua halaman setelah mendaftarkan URL-nya pada bagian *routing*.

Terakhir, tambahkan CSS berikut di bagian bawah `static/css/style.css`:

```

.experience-section {
    padding: 4rem 0;
    border-top: 1px solid var(--line);
}

.section-kicker {
    color: var(--accent-dark);
    font-size: 0.8rem;
    font-weight: 700;
    letter-spacing: 0.08em;
    text-transform: uppercase;
}

.experience-section h1 {
    margin: 0.4rem 0 1.5rem;
    font-size: 2rem;
}

.experience-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
    gap: 1rem;
}

.experience-card {

```

```
padding: 1.25rem;
background-color: #fff;
border: 1px solid var(--line);
border-radius: var(--radius);
}

.experience-category {
color: var(--accent-dark);
font-size: 0.75rem;
font-weight: 700;
text-transform: uppercase;
}

.experience-card h2 {
margin: 0.5rem 0;
font-size: 1.25rem;
}

.experience-description,
.empty-state {
color: var(--text-muted);
}

.experience-status {
margin-top: 1rem;
font-size: 0.85rem;
font-weight: 600;
}
```

`repeat(auto-fit, minmax(240px, 1fr))` membuat jumlah kolom menyesuaikan lebar layar. Kartu akan berjajar ketika ruang cukup dan turun menjadi satu kolom pada layar sempit.

Tutorial: Mengonfigurasi *Routing* URL

Routing adalah proses memetakan URL ke *view* yang akan menangani permintaan tersebut. Agar routing tetap terorganisasi, proyek akan meneruskan permintaan halaman profil dan pengalaman ke konfigurasi URL milik aplikasi `main`.

Langkah 1: Mengonfigurasi URL Aplikasi `main`

1. Buat berkas baru bernama `urls.py` di dalam direktori `main`.
2. Isi `main/urls.py` dengan kode berikut:

```
from django.urls import path

from main.views import show_main, show_experience

app_name = "main"

urlpatterns = [
    path("", show_main, name="show_main"),
```

```
path("experience/", show_experience, name="show_experience"),
]
```

Penjelasan Kode:

- `app_name = "main"` memberikan *namespace* pada URL milik aplikasi `main`.
- Pola URL `""` berarti halaman utama aplikasi tanpa tambahan path.
- Pola URL `"experience/"` mengarahkan permintaan `/experience/` ke `show_experience`.
- `name="show_main"` dan `name="show_experience"` memberi nama pada rute agar dapat dirujuk melalui navigasi dan test tanpa menulis URL secara langsung.

Langkah 2: Menghubungkan URL Proyek dengan URL Aplikasi

Buka `portofolio/urls.py`, lalu ubah isinya menjadi:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("main.urls")),
]
```

Fungsi `include()` meneruskan permintaan dengan awalan `""` dari level proyek ke `main/urls.py`. Di sana, Django memilih view berdasarkan sisa path. Rute langsung ke `landing_page` dari Tutorial 1 digantikan oleh `include("main.urls")`, sehingga halaman profil sekarang ditangani oleh `show_main`.

Kedua halaman mengikuti alur `Browser -> portofolio/urls.py -> main/urls.py`, lalu:

```
/                -> show_main          -> templates/index.html
/experience/     -> show_experience     -> templates/experience.html
```

`urls.py` proyek menghubungkan aplikasi, sedangkan `urls.py` aplikasi memilih view untuk setiap halaman. Satu aplikasi `main` dapat melayani kedua halaman ini.

Langkah 3: Menghubungkan Navigasi Antarhalaman

Buka **kedua berkas**, `templates/index.html` dan `templates/experience.html`. Pada masing-masing berkas, ganti elemen `<nav>...</nav>` dengan:

```
<nav>
  <a href="{% url 'main:show_main' %}">Profile</a>
  <a href="{% url 'main:show_experience' %}">Experience</a>
</nav>
```

Ubah pula tautan nama pada *header* kedua berkas agar selalu kembali ke halaman profil:

```
<a href="{% url 'main:show_main' %}" class="brand">{{ name }}</a>
```

Penjelasan Kode:

- `{% url 'main:show_experience' %}` menghasilkan URL berdasarkan *namespace* `main` dan nama rute `show_experience`, yaitu `/experience/` pada konfigurasi ini.
- `{% url 'main:show_main' %}` menghasilkan URL `/`, sehingga tautan **Profile** dan nama pada *header* dapat dipakai untuk kembali dari halaman pengalaman.
- Tautan `#profile` hanya berpindah ke elemen dengan `id="profile"` di halaman yang sedang dibuka. Untuk berpindah halaman, kita memakai URL rute yang sudah didaftarkan.

Langkah 4: Menjalankan Aplikasi

Jalankan pemeriksaan konfigurasi Django:

```
python manage.py check
```

Jika tidak ditemukan masalah, jalankan server pengembangan:

```
python manage.py runserver
```

1. Buka <http://localhost:8000/> pada *browser*. Halaman ini menampilkan profil dengan data dari `show_main`.
2. Klik **Experience**. Alamat browser berubah menjadi <http://localhost:8000/experience/>. Karena belum ada data, halaman ini menampilkan pesan **Belum ada pengalaman yang ditambahkan**. Kita akan mengisinya pada langkah berikutnya.
3. Klik **Profile** atau nama pada *header* untuk kembali ke halaman profil. Kartu pengalaman hanya tampil di halaman Experience.
4. Coba buka <http://localhost:8000/experience/> secara langsung untuk memastikan halaman tersebut juga dapat diakses tanpa melewati halaman profil.

Langkah 5: Menambahkan Data untuk Dicoba

Pada tahap ini, halaman Experience sudah dapat diakses, tetapi tabel `Experience` masih kosong. Hentikan server dengan `Ctrl+C`, lalu jalankan Django shell dari direktori yang berisi `manage.py` dengan *virtual environment* tetap aktif:

```
python manage.py shell
```

Masukkan satu pengalaman milikmu. Contoh berikut masih memakai data Burhan, jadi ubah isinya sebelum melanjutkan:

```
from main.models import Experience

Experience.objects.create(
    title="Asisten Dosen PBP",
    description="Membantu mahasiswa memahami dasar pengembangan web.",
    category="part-time",
)
```

Keluar dari shell dengan menjalankan `exit()`. Menambah data tidak membutuhkan `makemigrations`, karena struktur modelnya tidak berubah.

Jalankan kembali server dengan `python manage.py runserver`, lalu muat ulang <http://localhost:8000/experience/>. Data yang baru dibuat akan tampil sebagai kartu dengan kategori **Part-Time** dan status **Sedang berlangsung**.

Tip

Coba sedikit modifikasi: tambahkan satu objek `Experience` lagi dengan kategori berbeda, lalu ubah teks status atau warna kartunya. Pastikan kedua objek tetap tampil tanpa menulis kartu HTML kedua secara manual.

Tutorial: Pengenalan Django Unit Testing

Menjalankan aplikasi di *browser* membantu kita memeriksa tampilan, tetapi pemeriksaan manual mudah terlewat. *Unit test* membantu memastikan perilaku penting tetap benar setiap kali kode diubah.

Django menyediakan basis data khusus untuk pengujian. Data pengujian tidak masuk ke `db.sqlite3` milikmu dan akan dihapus setelah seluruh test selesai.

Langkah 1: Membuat Test

Saat membuat aplikasi `main`, Django sudah menyediakan berkas `main/tests.py`. Buka berkas tersebut, lalu ganti isinya dengan kode berikut:

```
from django.test import TestCase
from django.urls import reverse
from django.utils import timezone

from main.models import Experience

class MainTest(TestCase):
    def setUp(self):
        self.experience = Experience.objects.create(
            title="Asisten Dosen PBP",
            description="Membantu mahasiswa memahami pengembangan web.",
            category="part-time",
        )

    def test_main_url_is_accessible(self):
        response = self.client.get(reverse("main:show_main"))

        self.assertEqual(response.status_code, 200)
        self.assertTemplateUsed(response, "index.html")
        self.assertNotContains(response, self.experience.title)
        self.assertContains(response, f'href="{reverse("main:show_experience")}"')

    def test_nonexistent_page_returns_404(self):
        response = self.client.get("/halaman-yang-tidak-ada/")

        self.assertEqual(response.status_code, 404)

    def test_experience_model(self):
        self.assertEqual(str(self.experience), "Asisten Dosen PBP")
```

```

self.assertEqual(self.experience.category, "part-time")
self.assertTrue(self.experience.is_ongoing)

def test_experience_page(self):
    response = self.client.get(reverse("main:show_experience"))

    self.assertEqual(response.status_code, 200)
    self.assertTemplateUsed(response, "experience.html")
    self.assertContains(response, self.experience.title)
    self.assertContains(response, self.experience.description)
    self.assertContains(response, "Part-Time")
    self.assertContains(response, "Sedang berlangsung")
    self.assertContains(response, f'href="{reverse("main:show_main")}"')

def test_empty_experience_page(self):
    Experience.objects.all().delete()
    response = self.client.get(reverse("main:show_experience"))

    self.assertContains(response, "Belum ada pengalaman yang ditambahkan.")

def test_completed_experience(self):
    self.experience.ended_at = timezone.now()
    self.experience.save()
    response = self.client.get(reverse("main:show_experience"))

    self.assertFalse(self.experience.is_ongoing)
    self.assertContains(response, "Selesai")
    self.assertNotContains(response, "Sedang berlangsung")

```

Semua method yang namanya diawali `test_` akan dijalankan otomatis oleh Django. Method `setUp()` dijalankan sebelum setiap test sehingga tiap test memperoleh data awal yang bersih.

Enam test tersebut memeriksa hal yang berbeda:

1. Halaman profil dapat diakses, memakai `index.html`, tidak menampilkan kartu pengalaman, dan memiliki tautan ke halaman Experience.
2. URL yang tidak terdaftar menghasilkan status `404 Not Found`.
3. Model menyimpan nilai dan menghitung `is_ongoing` dengan benar.
4. Halaman Experience memakai `experience.html`, menampilkan data model beserta kategori dan statusnya, serta memiliki tautan kembali ke profil.
5. Halaman Experience menampilkan pesan yang sesuai ketika belum ada data.
6. Pengalaman dengan `ended_at` terisi menampilkan status **Selesai**.

`reverse()` mencari URL berdasarkan `app_name` dan `name` yang telah dibuat di `main/urls.py`, sama seperti tag `{% url %}` pada `template`. Cara ini membuat tautan dan test tetap mengikuti rute jika path berubah. `assertNotContains` memastikan teks tertentu tidak muncul dalam respons, sedangkan `timezone.now()` memberikan waktu saat ini untuk mengisi `ended_at` pada test pengalaman yang sudah selesai.

Langkah 2: Menjalankan Test

Hentikan server dengan `Ctrl+C` jika masih berjalan di terminal yang sama. Pastikan terminal berada di direktori yang berisi `manage.py` dan *virtual environment* masih aktif, lalu jalankan test untuk aplikasi `main`:

```
python manage.py test main
```

Jika semuanya benar, bagian akhir keluarannya akan menyerupai berikut:

```
.....
-----
Ran 6 tests in ...s

OK
```

Baris `.....` muncul tepat sebelum separator. Pada *verbosity* normal, setiap titik mewakili satu test yang berhasil. `F` mewakili *assertion* yang gagal, sedangkan `E` mewakili error.

Untuk output yang menampilkan nama setiap test, gunakan:

```
python manage.py test main --verbosity 2
```

Kamu juga dapat menjalankan semua test dalam proyek dengan `python manage.py test`.

Langkah 3: Membuktikan bahwa Test Dapat Menangkap Kesalahan

Test yang selalu hijau belum tentu benar-benar berguna. Mari buktikan bahwa test di atas dapat gagal ketika perilaku aplikasi tidak sesuai harapan.

1. Ubah sementara nilai `200` menjadi `201` pada `test_main_url_is_accessible`.
2. Jalankan kembali `python manage.py test main`.
3. Perhatikan pesan gagal yang menunjukkan `200 != 201`.
4. Kembalikan nilai tersebut menjadi `200`, lalu jalankan test sekali lagi.

Peringatan

Jangan lanjut sebelum test kembali menampilkan `OK`. Perubahan yang sengaja dibuat untuk memicu kegagalan tidak boleh ikut di-*commit*.

Penting

Menghubungkan ke Proyek Kamu - Ganti data profil dan `Experience` contoh dengan data milikmu.

Untuk fitur berikutnya, gunakan pola yang sama: buat model, ambil datanya di *view*, kirim lewat *context*, tampilkan dengan *template*, daftarkan URL-nya pada `urls.py` aplikasi, lalu lindungi perilaku pentingnya dengan test.

Pengumpulan

Tutorial ini dikumpulkan lewat slot submisi yang disediakan di SCELE, dalam bentuk **tautan ke commit** (bukan sekadar tautan repositori) di GitHub yang menunjukkan hasil akhir Tutorial ini, di-*push* **sebelum**

tenggat waktu di atas. Commit yang di-*push* setelah tenggat waktu tidak akan diterima, sehingga Tutorial ini dianggap belum selesai dan Individual Assignment minggu ini tidak akan dinilai. Pastikan juga repositori GitHub kamu bersifat **publik**, supaya asisten dosen bisa mengaksesnya.

Referensi Tambahan

- [Django - Writing views](#)
- [Django - URL dispatcher](#)
- [Django - The Django template language](#)
- [Django - Writing and running tests](#)